

Learning Objectives

Learners will be able to...

- Describe how text similarity metrics are calculated
- Calculate Euclidean distance between texts
- Calculate Jaccard Similarity between texts
- Calculate Cosine Similarity between texts

info

Make Sure to Know

Intermediate Python.

Limitations

In this assignment, we only work with the NLTK library.

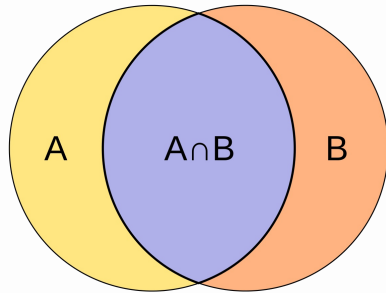
Jaccard Similarity Coefficient Score

There are various text similarity metrics that exist, such as Cosine similarity, Euclidean distance, and Jaccard Similarity.

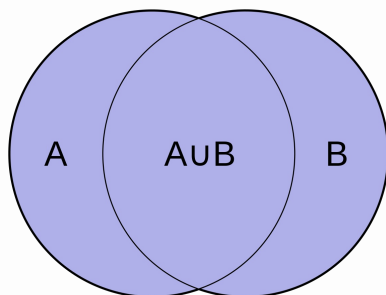
If we consider two documents, document A and document B, the Jaccard Score (sometimes called the similarity coefficient) is calculated as follows:

$$J(A, B) = \frac{A \cap B}{A \cup B}$$

The numerator or top part of the fraction is the **intersection** between the documents. In Jaccard, it is literally the count of words that are in both documents:



The denominator or bottom part of the fraction is the **union** between the documents. In Jaccard, it is literally the count of all unique words that are in either document:



Example calculation of Jaccard Score

Let's consider the following documents:

1. Bunnies like to eat lettuce more than carrots.
1. Fish like to play with bubbles while swimming.

Let's start by removing stop words, punctuation and capitalization:

1. bunnies like eat lettuce carrots
1. fish like play bubbles swimming

The **intersection** between these two documents is the word **like** which is 1 word.

The **union** between these two documents, or the count of unique words, is 9.

The Jaccard score is then calculated as follows:

$$J(A, B) = \frac{A \cap B}{A \cup B} = \frac{1}{9} \approx 0.1111$$

Programming Jaccard Score with `sklearn.metrics.jaccard_score`

To get started, we need to vectorize the text. Because `jaccard_score` is looking for integer vectors, we will use `CountVectorizer` instead of `TfidfVectorizer`:

```
vectorizer = CountVectorizer(stop_words='english', binary=True)
X = vectorizer.fit_transform(documents)
```

Then, we simply call `jaccard_score` on the vector representation of each document:

```
print(jaccard_score(X.toarray()[0], X.toarray()[1]))
```

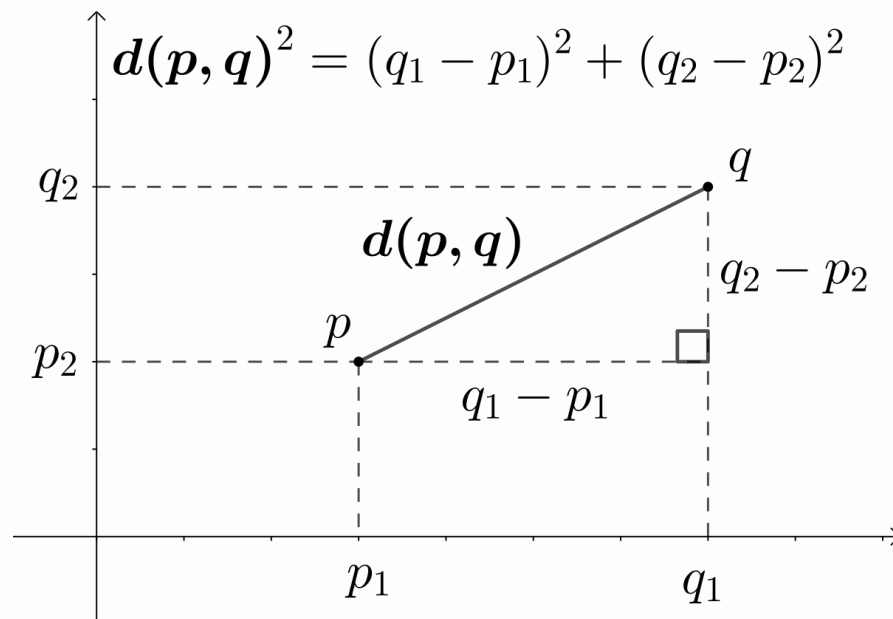
You should get `0.1111111111111111` - the same value we calculated above.

Try out different documents to see what their similarity score is!

Euclidean Distance

Euclidean distance is something you have probably seen in a math class - the basic idea behind it is Pythagorean's theorem ($a^2 + b^2 = c^2$ for the sides of a right triangle where c is the hypotenuse).

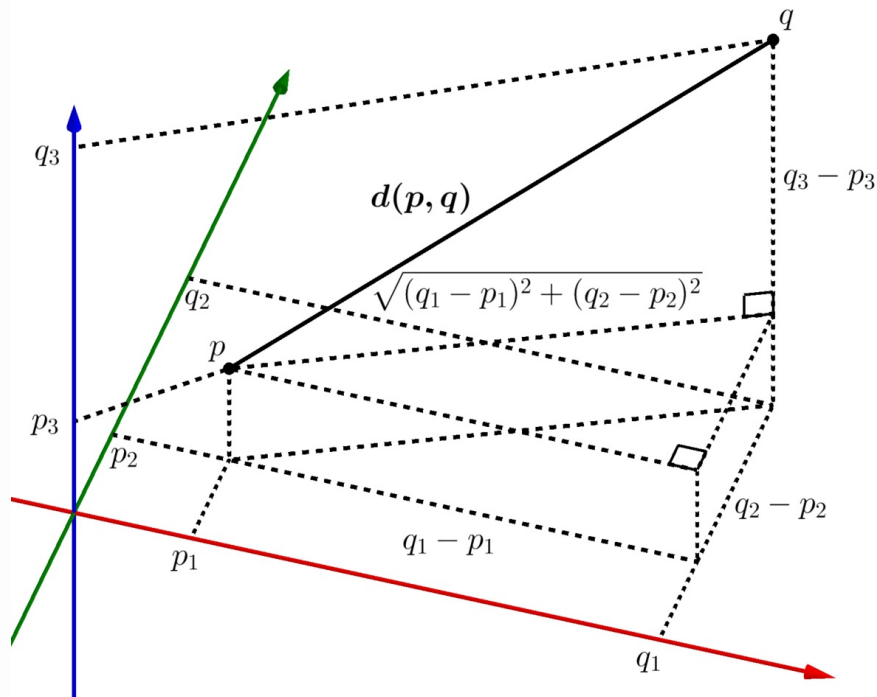
On a two-dimensional grid, this equation looks slightly different as each side of the right triangle is the difference along either the x or y dimension:



The resulting equation for distance in two dimensions is then:

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$

You can generalize this to three dimensions by using the resulting hypotenuse of the two-dimensional figure as one side and the third dimension as the second side:



The resulting equation for distance in three dimensions is then:

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + (q_3 - p_3)^2}$$

After 3 dimensions, distance gets hard to visualize, but you can generalize the equation to n dimensions:

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + (q_3 - p_3)^2 + \dots + (q_n - p_n)^2}$$

Example calculation of Euclidean Distance

Let's consider the following documents again:

1. Bunnies like to eat lettuce more than carrots.
1. Fish like to play with bubbles while swimming.

Let's start by removing stop words, punctuation and capitalization:

1. bunnies like eat lettuce carrots
1. fish like play bubbles swimming

Let's assume the following vocabulary:

```
['bubbles' 'bunnies' 'carrots' 'eat' 'fish' 'lettuce' 'like'
 'play' 'swimming']
```

We then have the following two 9-dimensional vectors to represent the documents:

```
[0, 1, 1, 1, 0, 1, 1, 0, 0]
[1, 0, 0, 0, 1, 0, 1, 1, 1]
```

$$d = \sqrt{1^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 1^2 + 1^2}$$

$$d = \sqrt{8} \approx 2.83$$

Programming Euclidean Distance with `sklearn.metrics.pairwise.euclidean_distances`

To get started, we need to vectorize the text just like before:

```
vectorizer = CountVectorizer(stop_words='english', binary=True)
X = vectorizer.fit_transform(documents)
```

Then, we simply call `euclidean_distances` on the vector representations:

```
print( euclidean_distances(X) )
```

You should see the following output:

```
[[0.          2.82842712]
 [2.82842712  0.          ]]
```

The way to read this is the top-left is the distance between the first vector and the first vector – so it makes sense that the distance to itself is 0.

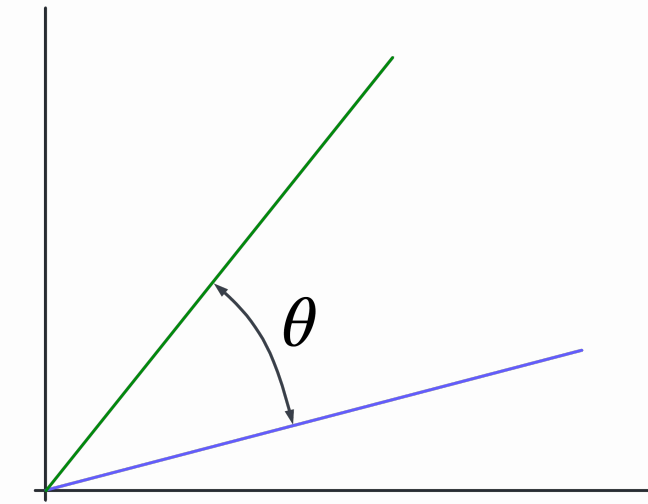
The top-right and bottom-left are both representing the distance between the first vector and the second vector. We see that the code got the same 2.83 number we calculated above.

The bottom-right number is the distance between the second vector and the second vector – so it makes sense that the distance to itself is 0.

This representation makes more sense when you are comparing more than two documents – try adding a third document!

Cosine Similarity

If Euclidean distance is the straight line distance between vectors, Cosine similarity is the angular distance between vectors:



.guides/img/cosine

The formula is:

$$\cos(\theta) = \frac{A \cdot B}{||x|| * ||y||} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Example calculation of Cosine Similarity

Let's consider the following documents again:

1. Bunnies like to eat lettuce more than carrots.
1. Fish like to play with bubbles while swimming.

Let's start by removing stop words, punctuation and capitalization:

1. bunnies like eat lettuce carrots
1. fish like play bubbles swimming

Let's assume the following vocabulary:

```
['bubbles' 'bunnies' 'carrots' 'eat' 'fish' 'lettuce' 'like' 'play' 'swimming']
```

We then have the following two 9-dimensional vectors to represent the documents:

```
[0, 1, 1, 1, 0, 1, 1, 0, 0]  
[1, 0, 0, 0, 1, 0, 1, 1, 1]
```

$$\begin{aligned} \cos(\theta) &= \frac{A \cdot B}{\|x\| * \|y\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \\ &= \frac{(0 * 1) + (1 * 0) + (1 * 0) + (1 * 0) + (0 * 1) + (1 * 0) + (1 * 1) + (0 * 1) + (0 * 1)}{\sqrt{0^2 + 1^2 + 1^2 + 1^2 + 0^2 + 1^2 + 1^2 + 0^2 + 0^2} \sqrt{1^2 + 0^2 + 0^2 + 0^2 + 1^2 + 0^2 + 1^2 + 1^2}} \\ &= \frac{1}{\sqrt{5} \sqrt{5}} = \frac{1}{5} = 0.2 \end{aligned}$$

Programming Cosine Similarity with `sklearn.metrics.pairwise.cosine_similarity`

To get started, we need to vectorize the text just like before:

```
vectorizer = CountVectorizer(stop_words='english', binary=True)
X = vectorizer.fit_transform(documents)
```

Then, we simply call `cosine_similarity` on the vector representations:

```
print( cosine_similarity(X) )
```

You should see the following output:

```
[[1.  0.2]
 [0.2 1.  ]]
```

The way to read this is the top-left is the cosine between the first vector and the first vector. Cosine ranges from 1 to -1 where $\cosine(0^\circ) = 1$ and $\cosine(180^\circ) = -1$ – so it makes sense that the cosine of a vector to itself is 1.

The top-right and bottom-left are both representing the cosine between the first vector and the second vector. We see that the code got the same 0.2 number we calculated above.

The bottom-right number is the cosine between the second vector and the second vector – so it makes sense that the cosine to itself is 1.

Formative Assessment 1

Formative Assessment 2